



Molecular
Quantum
Solutions

QPUBench

Open Source Quantum Computing Benchmark Framework

contact@mqs.dk

Molecular Quantum Solutions (MQS)

Framework Overview

What is QPUBench?

Modality-agnostic quantum benchmark framework — three layers, one record

Layer	Answers	Lives in
Schema	<i>What</i> are you benchmarking?	schemas/ — 37 Pydantic v2 modules (= “Schema v3.1.0”)
Adapter	<i>How</i> does it run?	backends/ + integrations/ (next slide)
Store	<i>How</i> is it persisted?	store.py — NDJSONStore, ParquetStore, S3Store

Core invariants

qpubench itself never imports from any quantum library — only adapters do. And since schema v3.0.0 the core schema modules import *zero vendor schema modules*: vendor data flows through vendor-neutral dict extension points (next section).

Architecture

Core layer (no SDK deps, pip install-able)

- schemas/ — Schema layer
- runner.py — BenchmarkRunner
- store.py — Store layer

Integration layer (reference code)

- backends/ — adapter protocols + stubs
- integrations/ — real reference adapters, **not installed** — copy into your project (like a template)
- examples/ — runnable demos



“**Contract**” = a Python Protocol — a fixed method signature (spec/validate/run) callers rely on without caring which class implements it. A **stub** satisfies it with fake data; real adapters (Aer, IBM, IQM, Braket) satisfy the same contract with real SDK calls. Swapping one for the other needs zero code changes elsewhere.

Key Design Principles

Principle	Implementation
Schema-first	Pydantic v2 models are the stable contract — every record round-trips through <code>.model_dump_json()</code>
No SDK lock-in	Core never imports any quantum library
Three adapter protocols	<code>BackendAdapter</code> · <code>AlgorithmAdapter</code> · <code>ErrorMitigationAdapter</code>
Append-only stores	<code>NDJSONStore</code> / <code>ParquetStore</code> for reproducible sweeps; <code>S3Store</code> for concurrent writers
Language-agnostic	Every record is plain JSON — any language can parse it, whichever store wrote it
Inputs vs. outputs	<code>VQAConfig</code> = what you chose to run; <code>VQAResult</code> = what the run produced — computed values are never user-supplied
Vendor-neutral core	Vendor payloads live in keyed dicts (<code>vendor_results</code> , <code>mitigation_options</code> , <code>vendor_data</code>) — adding a vendor never touches a core file

NDJSON vs. JSON, Precisely

Every BenchmarkRecord serializes to one JSON object, always. *How that object reaches disk* differs by store — same schema, three storage strategies, not an inconsistency:

Store	On-disk shape
NDJSONStore	One JSON object per line, appended to one shared file (“ N ewline- D elimited JSON”)
S3Store	One whole JSON file per record — no shared file, so concurrent writers can’t race
ParquetStore	Same records converted to a columnar table

Installation

Not yet on PyPI — install from a git clone until it is:

```
git clone https://github.com/mqsdm/qpubench && cd qpubench
pip install . # Minimal – schemas + stubs, no quantum SDK
pip install ".[qiskit]" # Qiskit Aer + IBM Quantum Runtime V2
pip install ".[storage]" # Parquet store (pyarrow + pandas)
pip install ".[s3]" # S3 store (boto3) – AWS S3 / MinIO / HF buckets
pip install ".[all]" # everything, all optional extras
```

Package managers: pip · uv · Poetry 2 · conda

Credentials: .env + BackendSpec.auth

Schema Layer

Reading This Reference

The next 9 slides are a **reference listing**, not a narrative — they exist so you know a module is there when you need it, not to be read top-to-bottom. The take-home message fits in one line:

Take-home message

Every schema module is a set of typed Pydantic models for one vendor/technique's inputs and outputs. 11 "core" modules are framework-owned (no vendor); the rest are one module per integration, named so the filename alone tells you who maintains it.

If you just want to see the package in action: skip ahead to *Adapters* (real running code) or *Examples & Coverage* at the end (a full worked example). Come back here when you need to look up what a specific module contains — that's what a GitHub repo's own reference docs (`docs/schemas.md`) are for; these slides just mirror the module list so it's not a total blank spot.

Core Schema Modules (1/3)

All rows are computing-model/qubit-modality **all** / **all** — these are the framework's own foundational types, not vendor-specific.

Module	Key types
primitives	ComputingModel, QubitModality, CircuitFormat, PauliLabel, ComplexNumber
circuit	CircuitSpec — from_openqasm3(), bind(), is_parametric()
observable	SparsePauliObservable, PauliTerm
backend	BackendSpec — 31 factory constructors

Core Schema Modules (2/3)

Module	Key types
execution	ExecutionOptions, AlgorithmSpec, ZNEConfig
advantage	QuantumAdvantageRecord — multi-org registry, unprefixed
result	QuantumResult (+ vendor_results dict), ExpectationResult, ShotResult
record	BenchmarkRecord, VQAConfig (inputs), VQAResult (computed outputs)

Core Schema Modules (3/3)

Module	Key types
optimizer_catalog	MINIMIZER_CATALOG, STOPPING_CRITERION_CATALOG — over AdaptVQEConfig fields
hamiltonian_library	HamiltonianSource, HamiltonianLibraryRecord — metadata for real Hamiltonians (PennyLane qchem, HamLib, ab initio)
contraction_path	ContractionPathStrategy/Config/Result — real quimb + cotengra

Domain-Specific Schema Modules (1/2)

Named `<org>_<package>.py` — who maintains the upstream project + what it's called.

Module	Computing model (qubit modality)
<code>johnrscott_mbqc_fpga</code>	MBQC-FPGA — bit-exact 16-bit program word
<code>mqsdk_photoq</code>	Photonic LOQC/FBQC + GBS — Fock states, HOM, photonic VQE/analog, hafnian, vibronic, TDM/Borealis, pseudo-PNRD methods, ORCA/QCloud/Aurora backends, BBS
<code>microsoft_qdk</code>	Gate-based (QPE technique) — SCF → active space → resource estimation
<code>mqsdk_qse</code>	Gate-based (KQD technique) — Krylov Quantum Diagonalization / SQD

Domain-Specific Schema Modules (2/2)

`classiq_classiq/pyscf_pyscf` double up their name because the maintaining org and the package name are the same word — kept for consistency rather than shortened. `polarizable_embedding` stays unprefixed: it bridges CPPE + PyFraME with no single vendor to name.

Module	Computing model (qubit modality)
<code>molssi_qcschema</code>	all / all — QCSchema / QCElemental / PennyLane interoperability
<code>quera_bloqade</code>	Adiabatic (neutral atom) — Bloqade / Aquila AHS
<code>erikkjellgren_slowquant</code>	Gate-based — SlowQuant UCC/VQE ansatz, SCF, linear response
<code>classiq_classiq</code>	Gate-based — Classiq synthesis + chemistry/QAOA
<code>pyscf_pyscf</code>	Gate-based (chemistry) — mean-field/DFT, PCM solvation, ERI builder
<code>polarizable_embedding</code>	Gate-based (chemistry) — CPPE + PyFraME, via <code>pyscf.solvent.PE</code>

Computing Model × Qubit Modality

Two **independent, orthogonal** axes — not one flat enum (a fixed set of named choices). Many-to-many: some paradigms run on several QPU modalities, but not every pairing exists.

ComputingModel (paradigm)

- GATE_BASED — circuit model
- MBQC — measurement-based
- FUSION_BASED — FBQC resource states + fusion
- ADIABATIC — continuous-time evolution (AHS)
- ANNEALING — quantum annealing
- GBS — Gaussian Boson Sampling
- SAMPLING — general sampling paradigms

QubitModality (QPU modality)

- SUPERCONDUCTING — IBM, IQM
- TRAPPED_ION — Quantinuum, IonQ
- NEUTRAL_ATOM — QuEra
- PHOTONIC — Quandela, Xanadu, ORCA, DTU
- SILICON_SPIN — Quantum Motion

GATE_BASED alone spans superconducting, trapped-ion, silicon-spin, and photonic hardware — GBS/FUSION_BASED are photonic-only today. QPE/KQD are techniques on GATE_BASED, not separate paradigms.

CircuitSpec — One Type for All Circuits

```
# Gate-based – read a .qasm file straight into `serialized`
source = pathlib.Path("bell_state.qasm").read_text()
circuit = CircuitSpec(num_qubits=2, serialized=source,      # OpenQASM 2.0
                      observables=[...])

# OpenQASM 3.0 – same file-read pattern
circuit = CircuitSpec.from_openqasm3(
    pathlib.Path("bell_state.qasm3").read_text(), num_qubits=2)

# Parametric VQE – bind() returns a new copy
ansatz = CircuitSpec(num_qubits=2, parameters=["theta"], ...)
bound   = ansatz.bind({"theta": 1.2566})
```

Schema v3.0.0 — Inputs vs Outputs, Vendor-Neutral Core

Two breaking changes define v3.0.0. **First:** VQA metadata is split — configs never carry computed values (passing one raises a `ValidationError`):

```
vqa = VQAConfig(problem_type="chemistry", molecule="H2", basis="sto-3g") # inputs
record = runner.run(bound_ansatz, "aer", shots=4096, vqa=vqa)
record.vqa_result.final_eigenvalue      # derived from expectation values by the runner
record.vqa_result.chemical_accuracy     # True (< 1.6 mHa) once a reference is present
```

Second: core schemas import zero vendor modules — vendor payloads travel as keyed dicts (pass a Pydantic model, it's auto-dumped; rehydrate with the vendor schema):

Extension point	Replaces	Example key
<code>QuantumResult.vendor_results</code>	24 vendor-typed result fields	"qforte_result", "slowquant_record"
<code>CircuitSpec.measurement_pattern / .photonic_circuit</code>	typed MBQC/photonic fields	<code>MBQCPattern.model_validate(...)</code>
<code>ExecutionOptions.mitigation_options</code>	QESEM-typed fields	<code>gesem_mitigation_options(...)</code>
<code>VQAConfig.vendor_data</code>	ad-hoc vendor fields	<code>{"cebutle": {...}}</code>

Real Hamiltonian Sources

Three ways to get a real molecule's qubit Hamiltonian as a `SparsePauliObservable` (`hamiltonian_sources.*`) — drops into any Estimator path:

Source	Function	What it gives you
HamLib Chemistry	<code>load_hamlib_chemistry()</code>	HDF5 (NERSC), 4 encodings, no ref. energies
PennyLane qchem	<code>load_pennylane_qchem()</code>	Dataset service, ships fci_energy/vqe_energy
Ab initio	<code>build_qubit_hamiltonian()</code>	<i>Any</i> geometry — PySCF HF + active-space + mapper

```
obs, record = build_qubit_hamiltonian(  
    geometry, basis="sto-3g", active_electrons=2, active_orbitals=2)
```

Verified: HamLib's H2 Hamiltonian through the *unmodified* ADAPT-VQE engine converges to -1.13146 Ha, matching dense diagonalization to 7e-15.

Tensor-Network & Chemistry Mechanisms

Real, verified mechanisms built on quimb+cotengra or PySCF (already a dependency):

Mechanism	Implementation	Verified
Contraction Path Finder	<code>quimb.tensor.Circuit + cotengra.HyperOptimizer</code>	4/4 strategies real; NONE costs more (180224 vs 248)
ERI Builder	<code>mol.intor('int2e')</code> vs <code>mf.density_fit()</code> (RI/DF)	RI matches standard ERIs to 2e-5 Ha (H2O/cc-pVDZ)
Gate Selector	GateSelector protocol — gradient screen vs full re-opt	Both converge to exact H2 ground state
Orbital Optimizer	Newton (CASSCF) / Simple / Basin-Hopping	All 3 agree on H2/6-31G to 3e-9 Ha

A real bug found along the way: `quimb's contraction_info(optimize=False)` raises `TypeError` — worked around via `opt_einsum.contract_path()` directly.

reactions — Bridging Cantera, PennyLane & Cebule

Moved out of “core” — it bridges three real external tools, not a shape qpubench invented:

Bridges to	What it adds
Cantera	<code>ReactionMechanism.to_cantera_yaml()</code> — real, loadable mechanism
PennyLane demo	<code>ReactionPathResult.rate_constant()</code> — barrier → classical $k = A \exp(-E_a/RT)$
Cebule <code>RXN_OPT</code>	Complementary: network flux optimization, not merged in

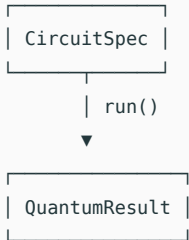
Verified against real cantera==3.2.0, all 3 reaction types. Real gotcha found: Cantera’s default quantity unit is kmol not mol — an unstated E_a silently becomes 1000x too small; `to_cantera_dict()` always declares units explicitly to prevent it.

Adapters

What vs. how: this slide is the *what* — three abstract shapes, no vendor code yet. Every slide after is the *how* — a vendor satisfying one shape with real code.

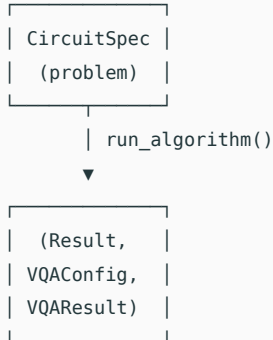
Three Adapter Protocols

BackendAdapter *Circuit in, result out*



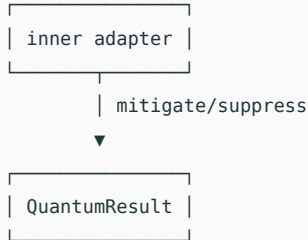
Aer · PennyLane · IBM · IQM
· Braket · MBQC

AlgorithmAdapter *Problem spec in*



QForte · SlowQuant · generic
ADAPT-VQE

ErrorMitigationAdapter *Wraps a BackendAdapter*



Mitiq (real, next section) Fire
Opal/Haiqu (schema only)

BenchmarkRunner dispatches automatically — ErrorMitigationAdapter registers like any BackendAdapter.

BackendAdapter — Estimator vs Sampler

```
class MyBackendAdapter:
    @property
    def spec(self) -> BackendSpec: ...    # hardware description
    def validate(self, circuit) -> list[str]: ...
```

Estimator path — circuit.observables set

```
def run(self, circuit, options):
    evs = [measure_expectation(circuit, obs)
           for obs in circuit.observables]
    return QuantumResult(
        expectation_values=evs, ...)
```

VQE / QAOA objective values

Sampler path — no observables

```
def run(self, circuit, options):
    counts = sample_bitstrings(
        circuit, options.shots)
    return QuantumResult(
        shots=ShotResult(
            ..., counts=counts), ...)
```

Raw bitstring / shot counts

Rule: SDK imports go inside the method that uses them — never at module level.

AlgorithmAdapter (QForte example)

```
class QForteAlgorithmAdapter:
```

```
    validate_problem()
```

```
def validate_problem(self, circuit):
    if circuit.format != \
        CircuitFormat.MOLECULE_JSON:
        return [
            f"Expected MOLECULE_JSON, "
            f"got {circuit.format!r}"
        ]
    return []
```

```
    run_algorithm()
```

```
def run_algorithm(self, circuit, options):
    import qforte as qf      # deferred import
    cfg = options.adapt_vqe_config
    mol = qf.system_factory(...)
    alg = qf.ADAPTVQE(mol, ...)
    alg.run(pool_type=cfg.pool_type,
            optimizer=cfg.optimizer)
    result = QuantumResult(
        vendor_results={"qforte_result":
            QForteRunResult(...)},
        expectation_values=[...])
    return (result, VQAConfig(...),
            VQAResult(final_eigenvalue=
                alg.get_gs_energy(), ...))
```

AlgorithmFamily — Switch Implementations Freely

AlgorithmSpec carries **identity only** (name + family) — hyperparameters live in one shared config every adapter for that family accepts:

```
config = AdaptVQEConfig(pool_type="SD", optimizer="BFGS")
opts = ExecutionOptions(algorithm_spec=AlgorithmSpec(
    name="ADAPT_VQE", family=AlgorithmFamily.ADAPT_VQE),
    adapt_vqe_config=config)

runner.run(mol, "qforte", opts)           # native C++
runner.run(mol, "ibm_qiskit_adapt_vqe", opts)  # from-scratch Qiskit
runner.run(mol, "microsoft_qdk_adapt_vqe", opts) # QDK / Azure Quantum
```

Three adapters implement AlgorithmFamily.ADAPT_VQE: integrations/qforte/ (native C++), and ibm_qiskit_adapt_vqe//microsoft_qdk_adapt_vqe/ (thin wrappers over the pure-Python generic_adapt_vqe/ engine).

Circuit Optimization — Three Real Strategies, Not Two

Every gate-based backend adapter already runs the **Qiskit transpiler** (`ExecutionOptions.optimization_level 0–3 + TranspilerConfig`) — the baseline every circuit gets. Xenakis/Classiq below are two *additional*, heavier strategies for when that isn't enough.

	Qiskit transpiler	Xenakis	Classiq
Strategy	Preset pass manager	GA search	Constrained synthesis
Objective	<code>optimization_level</code> 0–3	Soft penalty ($\lambda_{\text{depth}}, \lambda_{2q}$)	Hard bound + objective
Cost	One pass, ms	Generations $\times$ population	One solve (duration, s)

Not yet modeled: Microsoft QDK/Q#'s own circuit-optimization passes have no schema representation here — `microsoft_qdk.py` stops at resource estimation. Open gap, not silently assumed.

Xenakis ↔ Classiq — Comparing Two Optimization Runs

```
classiq_mol = ClassiqMoleculeSpec.from_xenakis_molecule(xenakis_mol)
cmp = CircuitOptimizationComparison(ga_result=ga_run, classiq_result=synth)
cmp.depth_delta          # GA best-genome depth - Classiq depth
cmp.search_cost_label    # "GA: 40 generations vs Classiq: 2.30s synthesis"
```

Error Mitigation Providers

One module per vendor, one shared `ErrorMitigationStrategy` enum in `primitives.py`:

Provider	Schema module	Technique
Q-CTRL Fire Opal Mitiq	<code>qctrl_fire_opal</code> <code>unitaryfund_mitiq</code>	Closed-box noise suppression ZNE · PEC · CDR · REM · DDD
Haiqu Rivet ParityQC	<code>haiqu_rivet</code> <code>parityqc_parityqc</code>	HW-aware transpilation caching Parity encoding (QUBO/HCBO/Ising)
QMatter Quantum Motion	<code>qmatter_qmatter</code> <code>quantum_motion_hardware</code>	Quantum problem compression CMOS spin-qubit characterisation (<i>hardware vendor</i>)
QESem (Qedma)	<code>qedma_qesem</code>	QET noise scaling + characterisation

ErrorMitigationAdapter — Demonstrated (Mitiq ZNE)

Only **Mitiq** has a real ErrorMitigationAdapter behind it today (MitiqZNEAdapter) — the rest on the previous slide are schema-only metadata. Wraps any BackendAdapter; mitigates the estimator path only.

```
noisy_inner = AerAdapter(noise_model_json=depolarizing_2pct_5pct_json)
raw = noisy_inner.run(bell_circuit, ExecutionOptions())
raw.expectation_values[0].value          # 0.950  (noisy)

adapter = MitiqZNEAdapter(noisy_inner)    # same BackendAdapter contract
mitigated = adapter.run(bell_circuit, ExecutionOptions())
mitigated.expectation_values[0].value     # 0.997  (extrapolated)
```

Verified against a real depolarizing-noise Aer simulator (2%/5% error on H/CX): ZNE recovers 0.997 vs. raw noisy 0.950 and noiseless-exact 1.0 — real Mitiq `Factory.run().reduce()` calls, the adapter `base.py` described but that never existed until now.

Running on Real Hardware — IBM, IQM & AWS Braket

All three adapters are **real, verified end-to-end** — Aer/Braket need no credentials (BraketLocalBackend); IBM/IQM verified against bundled fake backends. `opts = ExecutionOptions(shots=4096)` shared by all three.

IBM Quantum Runtime V2

```
adapter = IBMAdapter(  
    "ibm_torino",  
    execution_mode=  
        IBMExecutionMode.SESSION)  
runner.register(adapter, "ibm")  
runner.run(circuit, "ibm", opts)
```

Estimator/Sampler auto-chosen.

IQM Resonance cloud

```
adapter = IQMAdapter("garnet",  
    token=os.environ["IQM_TOKEN"])  
runner.register(adapter, "iqm")  
runner.run(circuit, "iqm", opts)
```

Star topology, no SWAP routing.

AWS Braket

```
spec = BackendSpec.braket(  
    "arn:...Ankaa-3",  
    s3_bucket_ref="BUCKET")  
adapter = BraketAdapter(  
    spec.auth["device_arn"],  
    s3_bucket_ref="BUCKET")  
runner.register(adapter, "braket")
```

Needs an S3 result location.

All three implement `TranspiledBackend`. Two real SDK migrations found while verifying: `qiskit-iqm` standalone is obsolete (`iqm-client[qiskit]` replaces it); IBM's old `channel="ibm_quantum"` is gone (`"ibm_quantum_platform"` replaces it).

IBM Quantum Advantage Tracker

Problem: “quantum advantage” claims are hard to compare across papers — no shared record of what ran, on what hardware, vs. what classical method. `BenchmarkRecord.advantage` holds one `QuantumAdvantageRecord`, attached after the fact:

```
record = runner.run(circuit, "ibm_torino", options)
record = record.model_copy(update={"advantage": QuantumAdvantageRecord(
    experiment_type=AdvantageExperimentType.OBSERVABLE_ESTIMATION,
    circuit_name="Loschmidt-echo-70q", num_qubits=70, backend_name="ibm_boston",
    observable_value=0.327, observable_error_bound=0.012,
    classical_method=ClassicalComparisonMethod.TENSOR_NETWORK,
    submission_url="https://github.com/quantum-advantage-tracker/...",
}))
```

Quantum Advantage Tracker: community registry co-initiated by IBM, Flatiron Institute, BlueQubit, Algorithmiq. Three experiment categories: observable estimation · variational · classically verifiable.

Execution & Storage

BenchmarkRunner

```
runner = BenchmarkRunner(store=NDJSONStore(Path("results.ndjson")))
runner.register(AerAdapter(), name="aer")
runner.register(QForteAlgorithmAdapter(), name="qforte")
record = runner.run(circuit, "aer", ExecutionOptions(shots=4096)) # single run

# Cartesian product sweep: circuits x backends x options
records = runner.sweep(
    circuits=[circuit_h2, circuit_lih], backend_names=["aer", "qforte"],
    options_list=[ExecutionOptions(shots=1024), ExecutionOptions(shots=8192)],
    run_id="vqe-sweep-001",
)
runner.add_hook(lambda rec: log_energy(rec.vqa_result.final_eigenvalue)) # post-run hook
```

Structured Logging — BenchmarkLogger

Built directly on `BenchmarkRunner.add_hook()` — real levels, handlers, and formatters, not one flat log line:

```
from qpubench import BenchmarkLogger
bench_logger = BenchmarkLogger(name="qpubench.benchmark")
bench_logger.attach(runner)          # wires into add_hook() internally
record = runner.run(circuit, "aer", ExecutionOptions(shots=4096))
# {"experiment_id": "...", "backend": "aer_statevector", "status": "succeeded", ...}
```

Levels	Handlers	Formatters
SUCCEEDED → INFO, FAILED → ERROR	any logging.Handler	JSONFormatter (default), swappable

Data Persistence

NDJSONStore — zero dependencies, append-only, grep-able

```
store = NDJSONStore(Path("results/run1.ndjson"))
store.save(record)
record = store.load(experiment_id)           # by UUID
records = store.query(backend__name="aer_statevector", result__status="succeeded")
```

ParquetStore — columnar analytics (pip install "[storage]")

```
store = ParquetStore(Path("results/run1.parquet"))
df = store.to_dataframe()                   # → pandas DataFrame
```

Every record is one JSON line — stream-able, pandas-ready, archive-safe.

S3Store — AWS S3 & Hugging Face Buckets

One JSON object per record (`{prefix}/{experiment_id}.json`) — no shared file, so concurrent writers never race. pip install `"qpbench[s3]"`.

AWS S3 (S3-compatible too)

```
store = S3Store(  
    "my-results-bucket",  
    prefix="sweeps/2026-07-06",  
    region_name="eu-west-1",  
)
```

Any `boto3.client("s3", ...)` kwarg works — same path covers MinIO too.

Same `ResultStore` protocol as `NDJSONStore`: `save()` / `load()` / `query()` / `all()`.

Hugging Face Buckets

```
store = S3Store.huggingface(  
    bucket="my-training-bucket",  
    namespace="my-username",  
    access_key_id="HFAK...",  
)
```

`.huggingface()` sets the gateway's required Config.

Integrations

Integration Ecosystem — BackendAdapter

Schema modules are named `<org>_<package>.py` — one glance tells you who maintains it and what it's called.

Integration	Schema module
Cebule SDK	<code>mqsdk_cebule</code>
photoq (photochipsim, DTU-GBS, photonic_QC, pseudo-PNRD, ORCA/Xanadu/DTU backends)	<code>mqsdk_photoq</code>
QESEM (Qedma)	<code>qedma_qesem</code>
Bloqade / Aquila	<code>quera_bloqade</code>
IQM Resonance (garnet · deneb · sirius)	—
Quantum Motion CMOS spin-qubit	<code>quantum_motion_hardware</code>

Integration Ecosystem — AlgorithmAdapter

Integration	Schema module
Xenakis GA circuit search	mqsdk_xenakis
QForté (ADAPT-VQE, UCCN-VQE, ...)	evangelistalab_qforte
Classiq circuit synthesis	classiq_classiq
ExcitationSolve (Fourier VQE)	dlr_excitation_solve
GSOpt (ground-state benchmark)	bestquark_gsopt
QDK / QuNorth chemistry	microsoft_qdk
QSE / KQD	mqsdk_qse
SlowQuant UCC/VQE (integrations/slowquant/, real code, not on PyPI)	erikkjellgren_slowquant

Integration Ecosystem — ErrorMitigationAdapter

One module per vendor now — previously all bundled into a single `error_mitigation` grab-bag (including a hardware vendor miscategorized as “error mitigation”).

Integration	Schema module
Q-CTRL Fire Opal	<code>qctrl_fire_opal</code>
Mitiq (ZNE · PEC · CDR · REM · DDD)	<code>unitaryfund_mitiq</code>
Haiqu Rivet transpiler	<code>haiqu_rivet</code>
ParityQC parity encoding	<code>parityqc_parityqc</code>
QMatter problem compression	<code>qmatter_qmatter</code>

What Is Kubeflow, and Why Is It Here?

For readers who haven't met Kubeflow: an open-source platform that runs a workflow as a graph of steps (a DAG) on Kubernetes — each step is a container; it schedules them, tracks inputs/outputs, and shows the graph in a web dashboard.

```
mol_map → tn_qc_opt → qasm_gen → execute_circuits
  (box)      (box)      (box)      (box)
```

Why it's here: earlier integrations only needed `runner.run()`. Kubeflow becomes useful once a *sequence* of qpubench steps needs to run unattended, on a schedule, with a visual record — e.g. a nightly benchmark sweep. The next slides show that sequence as a real, compiled Kubeflow pipeline.

Orchestration — Kubeflow Pipelines

Different from every integration above: not a schema bridge, but how algorithmic steps get **run** as scheduled, dashboard-visible components.

Two Kubeflow SDKs, two roles:

SDK	Role here
kfp (Pipelines)	Owns the DAG + Central Dashboard — every step is a native <code>@dsl.component</code>
Unified SDK	Narrow, opt-in (<code>TrainerClient</code> / <code>OptimizerClient</code>) — called <i>inside</i> one component body only, for multi-node dispatch or Katib search

Why not the unified SDK as the top-level abstraction?

Its own docs list Pipelines support as "Planned," not shipped, and `TrainerClient`/`CustomTrainer` are shaped around ML training. `kfp` already gives DAG construction, branching, and caching natively.

Component Taxonomy

Every pipeline decomposes into two kinds of kfp node — same shape, different resource profile:

Kind	Shape	Examples
Transform	pydantic-in → pydantic-out, no BackendAdapter	SCF, active-space selection, Cebule MOL_MAP/TN_QC_OPT
Execution	BenchmarkRunner.run() against a registered adapter	GPU sim (Qrack) · real QPU (IBM, IQM, QESEM)

Kubeflow can't reach a QPU directly — an Execution component does what BackendAdapter.run() already does: call the vendor SDK, submit to *their* queue, poll, return. Kubeflow's role is a scheduled, credentialed place to host that client, not compute.

Iterative loops stay one job — `n_iterations/macro-micro-iterations` never explode into one DAG node per iteration.

Worked Example — Cebule Task Chain as Four kfp DAGs

integrations/kubeflow/ — one @dsl.pipeline per Mapper category in data/IBM VQE Test Benchmark.csv, matching Cebule SDK's own documented task order:

```
tn_qc_opt+mol_map: mol_map → tn_qc_opt → qasm_gen → execute_circuits
tn_qc_opt:          jw_map → tn_qc_opt → qasm_gen → execute_circuits
mol_map:            mol_map ────────────▶ execute_static_hamiltonian
JW:                 jw map ───────────▶ execute static hamiltonian
```

mol_map/JW carry no VQE ansatz — they measure the fixed Hartree-Fock reference state instead.

Verified by actually compiling all four against kfp==2.16.1: surfaced a real gotcha — from `__future__` import annotations breaks kfp's component introspection (needs live type objects, not PEP 563 strings). Every component body was also run directly (`.python_func`), including a real PySCF/OpenFermion `jw_map` call. Credentials injected via kfp-kubernetes's `use_secret_as_env`, never as plain pipeline parameters.

Kubeflow — Sweep Design

Rule: Mapper change → new pipeline (different components). Every other benchmark column is a *parameter value*, resolved inside one pipeline run — never one DAG per CSV row:

Column	Resolved via
TN_Layers_Network/Circuit, Rotation_Type, Measurement_Method	nested <code>dsl.ParallelFor</code> — one dashboard graph, fanned out
Shots, Qiskit_Opt_Level	<code>BenchmarkRunner.sweep()</code> <i>inside</i> the Execution component — same “stays one job” treatment as <code>n_iterations</code>

Conversion utilities — closed TODO

`SparsePauliObservable.to_dense_matrix()` / `.from_dense_matrix()` now ship in core (verified against Qiskit's `SparsePauliOp`, size-guarded via `max_qubits`) — all 6 (`hamiltonian_kind`, `measurement_method`) branch combinations run; only the live Cebule `create_task` calls still await real credentials.

Cross-Framework Compatibility

Pauli encoding — explicit converters prevent silent convention bugs:

```
# Qrack Q# convention: I=0, X=1, Z=2, Y=3 (non-sequential!)
PauliLabel.Z.to_qrack_int()          # → 2 (not 3)
PauliLabel.Y.to_qrack_int()          # → 3 (not 2)

# Qiskit C API QkBitTerm bit-packed encoding
PauliLabel.X.to_qiskit_c_bit_term()  # → 0b0010

# MBQC byproduct register: bit 0 = Z, bit 1 = X (reversed vs gate-based!)
```

Complex numbers: stored as {"re": 1.0, "im": 0.5} — valid JSON without Python eval

QCSchema / QCElemental / PennyLane interoperability via the `molssi_qcschema` module

Examples & Coverage

Guides, Demos & Tutorials Supported

examples/ — 30+ runnable examples built only on qpubench's own schemas and adapters:

Category	Count	Notes
Guides	20	Each maps to a real qpubench mechanism
Demos	6	End-to-end scripts combining several mechanisms
Tutorials	5	Real molecules, via the open ADAPT-VQE algorithm
Top-level	3	Gate-based, MBQC, and QForte VQE quick starts

Full breakdown — what's fully supported vs. partial, and why — in `examples/README.md`.

One Guide, End to End

examples/guides/vqe_calculator.py — all three layers in one runnable file:

```
hamiltonian = toy_hamiltonian()                                # Schema layer
config = AdaptVQEConfig(pool_type="SD", optimizer="BFGS", gradient_threshold=1e-5)

calculator = GenericAdaptVQEEngine(                             # Adapter layer
    hamiltonian=hamiltonian, num_qubits=4, num_electrons=2,
    energy_backend=ToyStatevectorAdapter(), config=config,
)
result, vqa, vqa_result = calculator.run()

# Store layer (elsewhere, same as every example): NDJSONStore(...).save(...)
```

Real output: HF: -4.000000 → ADAPT-VQE (3 iters): -4.472136 → exact: -4.472136 (matches to 1e-12) — the same three-layer shape as every guide/demo above.

Methodology — Honest by Construction (1/2)

No fabricated chemistry

Illustrative qubit Hamiltonians (`toy_hamiltonians.py`) stay explicitly labeled, never presented as physically accurate. Real molecules: `qforte_vqe_benchmark.py` (He/cc-pVDZ) and `hamiltonian_sources/` — real ab initio Hamiltonians replaced toy ones in 3 tutorials, checked against real reference notebooks.

No random numbers standing in for physics

`StubGateAdapter` returns *random* expectation values — useless as an ADAPT-VQE energy oracle. Built `ToyStatevectorAdapter`: a real, small, dense-matrix statevector simulator instead.

Methodology — Honest by Construction (2/2)

No fabricated API calls either

PsiEmbed, libDMET, and SlowQuant aren't on PyPI. Their adapters call each package's *actual documented API* — checked field-by-field against real GitHub source — behind an ImportError guard: real code, honestly unexecuted until installed from source.

```
# HF reference:          -4.000000  
# ADAPT-VQE (3 iters):   -4.472136  
# exact diagonalization: -4.472136  <- matches to 1e-12
```

Summary

Getting Started

```
from qpubench import (BenchmarkRunner, NDJSONStore, StubGateAdapter,
                      CircuitSpec, ExecutionOptions, SparsePauliObservable)
import pathlib

circuit = CircuitSpec(
    num_qubits=2, serialized=pathlib.Path("bell_state.qasm").read_text(),
    observables=[SparsePauliObservable(num_qubits=2, terms=[...])],
)
runner = BenchmarkRunner(store=NDJSONStore(pathlib.Path("results.ndjson")))
runner.register(StubGateAdapter(seed=42), name="stub")
record = runner.run(circuit, "stub", ExecutionOptions(shots=4096))
print(f"<ZZ> = {record.result.expectation_values[0].value:.4f}")
```

Engineering Quality Gates

Every push to main and every PR runs the full gate set in CI (`.github/workflows/ci.yml`, Python 3.11 + 3.12):

Gate	State
<code>pytest tests/</code> <code>ruff check</code>	340+ tests, no quantum SDK required 0 errors (deliberate lazy imports allowlisted per-file)
<code>mypy --strict src/qpubench</code>	0 errors (untyped optional SDKs silenced per-module)
Doc/code consistency	<code>tests/test_docs_consistency.py</code> fails CI if the schema version or module count drifts between code and docs

Tracking document

`CODE_QUALITY_REVIEW.md` — living record of resolved findings, open Todos, and conventions (single-source schema version, vendor extension points, no computed values in configs).

Roadmap & Links

Repository: github.com/mqsdk/qpubench

Schema v3.1.0 — 37 modules · 7 computing models × 5 qubit modalities · zero quantum SDK deps in core

Contributing — writing an adapter takes ~30 lines:

```
cp integrations/template/backend_adapter_template.py my_adapter.py
# fill the three TODOs: spec, validate(), run()
```

See [INTEGRATION_GUIDE.md](#) and [AGENTS.md](#) for the full development workflow.

Contact: contact@mqs.dk